

Bluechip Recommendation Agent Paper

DSN x BCT LLM Agent Challenge - Task B: Personalized Recommendation Agent

Abstract

Bluechip's recommendation agent is a retrieval-first, evidence-grounded system for personalized product recommendation. It handles sparse users, cross-domain contexts, explicit conversational context, and explanation generation through a multi-head retrieval and hybrid ranking pipeline. The LLM is not the recommender. It is an optional explanation layer after retrieval and ranking have already made the measurable decision.

The current Task B evidence shows the main bottleneck honestly: candidate recall. The strongest logged 100-example/1000-candidate gate has candidate Recall@1000 0.34, HitRate@10 0.10, and NDCG@10 0.0766. Cross-domain candidate Recall@1000 is the strongest measured slice at 0.5484. A later candidate-recall fast proof showed that positive-recommendation target alignment improves retrieval: Recall@1000 0.3986, sparse Recall@1000 0.3973, and cross-domain Recall@1000 0.6081. We do not claim that the final ranker has been promoted on this target until same-target HitRate@10 and NDCG@10 are run.

1. Task Framing

Task B is a recommender-system problem with an agent interface. A useful agent must understand user evidence, retrieve plausible products, rank them, explain why they fit, and expose enough diagnostics to debug failures.

Bluechip uses this serving contract:

```
user history + persona + current context
-> user profile
-> candidate retrieval
-> hybrid ranking
-> grounded explanations
-> diagnostics and trace
```

The key design principle is retrieval before ranking. If the correct item is not in the candidate pool, no ranker or LLM explanation can recover it. This is why the evaluation separates candidate Recall@K from HitRate@10 and NDCG@10.

2. Retrieval Architecture

The agent uses multiple retrieval heads because no single source works across dense, sparse, cold-start, and cross-domain users.

Retrieval head: Purpose

Co-visitation / item-to-item: Finds items connected through shared user behavior.

User-neighbor collaborative retrieval: Uses similar users where enough behavior exists.

Review-term retrieval: Matches user aspect language to product evidence.

Lexical item-neighbor retrieval: Finds title and metadata neighbors.

Evidence graph retrieval: Connects user aspects, categories, and item evidence.

BM25 profile/context retrieval: Makes current intent and persona text affect retrieval.

Category-affinity popularity: Provides robust candidates for sparse users.

Global popularity fallback: Prevents empty or brittle candidate sets.

Neural FAISS diagnostics: Confirms vector index wiring, but is not claimed as a quality win yet.

Candidate source attribution is preserved through the pipeline. This lets the evaluator report which heads found the held-out item and which heads only added noise.

3. Ranking and Explanation

After retrieval, Bluechip applies a hybrid scorer with explicit components:

- preference match
- context match
- category match
- aspect match
- sequential signal
- evidence graph signal
- Nigerian-context match
- collaborative signal
- retrieval match
- source diversity
- popularity
- novelty
- item quality
- confidence
- penalties for disliked or context-mismatched evidence

Explanations are generated after ranking. The explanation layer uses the ranked item, profile evidence, item evidence, and score components. This prevents the LLM from inventing a recommendation that the ranking system did not select.

4. Slice-Aware Recommendation Behavior

Sparse and dense users need different retrieval mixes. Dense users can rely more on collaborative and item-to-item behavior. Sparse users need stronger category, lexical, aspect, and popularity fallbacks. Beauty and other text-heavy categories benefit from review-term and aspect evidence

because broad popularity alone misses many intent-specific products.

The current implementation includes the machinery for this slice-aware behavior. The fast proof also showed that target alignment matters: when the task is framed as positive recommendation rather than every next interaction, retrieval improves on the slices that matter most for recommendation quality.

5. Implementation

Important local modules:

Area: Implementation

API: app/api/routes.py, app/main.py

Task B orchestration: app/serving/orchestrators/recommendation.py

Candidate retrieval: app/services/retrieval/candidates.py, app/services/retrieval/source_registry.py

Item similarity: app/services/retrieval/item_similarity.py

Context intent: app/services/ranking/context_intents.py, eval/task_b_context.py

Ranking: app/services/ranking/

Optional learned ranker: app/services/ranking/learned_task_b.py, eval/train_task_b_ranker.py

Evaluation: eval/eval_task_b.py, eval/aggregate_task_b_reports.py, eval/report_task_b_from_row_cache.py

Human-eval packet: eval/create_task_b_contextual_eval.py

Traces and diagnostics: /api/metrics, /api/traces

The optional learned ranker is gated. It is implemented, but the submission does not claim final lift because the full same-target ranker run was not completed before submission.

6. Evaluation Evidence

Current bounded Task B metrics:

Metric: Value - Interpretation

hybrid_candidate_recall@50: 0.13 - Early candidate pool remains weak.

hybrid_candidate_recall@100: 0.18 - Recall is the main bottleneck.

hybrid_candidate_recall@1000: 0.34 - Best current overall candidate recall signal.

hybrid_ranker_hit_rate@10: 0.10 - Current top-10 ranking gate.

hybrid_ranker_ndcg@10: 0.0766 - Current top-10 ranking gate.

Sparse candidate recall@1000: 0.3611 - Sparse-user handling is credible but incomplete.

Cross-domain candidate recall@1000: 0.5484 - Strongest measured Task B slice.

Vector source recall: 0.0 - Diagnostic only, not a promoted quality signal.

Final 50-example all-category smoke at candidate-limit 100:

Metric: Value

HitRate@10: 0.06

NDCG@10: 0.0471

Candidate Recall@50: 0.08

Candidate Recall@100: 0.12

Cross-domain candidate Recall@100: 0.2857

24 May 2026 candidate-recall fast proof:

Target mode: Key result

All interactions: Recall@1000 improved to 0.362, cross-domain Recall@1000 to 0.5752, but Recall@100 fell to 0.1589.

Positive recommendation: Recall@50 0.151, Recall@100 0.1823, Recall@1000 0.3986, sparse Recall@1000 0.3973, cross-domain Recall@1000 0.6081.

This evidence supports the direction of the engineering work: target alignment and multi-head candidate retrieval help. It does not yet support claiming a final top-10 ranking promotion.

7. Nigerian Contextualization

For recommendations, Nigerian context affects score components and explanations when relevant. The agent can use budget sensitivity, delivery reliability, authenticity concerns, city cues, market/mall distinctions, and social proof. These are not added as generic flavor. They contribute when the user's persona or request makes them decision-relevant.

Example contextual intents include affordable gifts, reliable delivery in Lagos or Abuja, durable business inventory, original-versus-fake quality concerns, and value-for-money choices.

8. Reproducibility

Core local commands:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
uvicorn app.main:app --reload
```

Task B evaluation commands:

```
python eval/eval_task_b.py
python eval/aggregate_task_b_reports.py
python eval/report_task_b_from_row_cache.py
python eval/create_task_b_contextual_eval.py
```

Useful URLs:

```
http://127.0.0.1:8000/ui/
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/api/health
http://127.0.0.1:8000/api/metrics
http://127.0.0.1:8000/api/traces
```

9. Limitations and Next Work

Known limitations:

- Candidate Recall@50 and Recall@100 are still too low.
- HitRate@10 and NDCG@10 are modest and should be treated as baseline gates.
- Positive-recommendation retrieval improved recall, but same-target top-10 ranker metrics remain next work.
- Neural/vector retrieval is wired and reproducible, but not yet a promoted quality signal.
- Local retrieval artifacts are production-shaped, not managed vector or graph services.

Highest-value next work:

1. Promote the positive-recommendation target only after same-target HitRate@10 and NDCG@10 beat the current 0.10 / 0.0766 gate.
2. Improve candidate Recall@50 and Recall@100 before investing more in ranker complexity.
3. Make candidate mixing more slice-aware for sparse users and Beauty-like text-heavy categories.
4. Train pairwise or listwise rankers with hard negatives once candidate recall is high enough.
5. Move retrieval artifacts to managed vector, lexical, and graph services for production scale.

10. Rubric Alignment

Brief requirement: Recommendation agent response

Personalized recommendations: /api/recommend retrieves, ranks, explains, and returns diagnostics.

Cold-start and sparse users: Persona, category, lexical, aspect, and popularity fallbacks are implemented.

Cross-domain behavior: Cross-domain candidate Recall@1000 is the strongest measured slice at 0.5484; positive target proof reached 0.6081.

Multi-turn scenarios: Conversation and feedback endpoints maintain context.

Explainability: Reasons are grounded in user evidence, item evidence, and score components.

Reproducibility: Runs locally without secrets and exposes deterministic fallbacks.

Honest evaluation: Candidate recall, ranking metrics, source attribution, and limitations are separated.

Bluechip's recommendation-agent claim is intentionally practical: improve retrieval first, prove top-10 lift second, and let the LLM explain the ranked decision rather than make it.